

RaftMessaging: Project Written Integration

Due on March 19, 2026

Soldà Matteo

March 19, 2026

Snapshotting Lock

During the testing phase of the project, it became evident that the snapshotting mechanism, although executed asynchronously, introduced significant bottlenecks due to the use of a global lock. This behavior severely impacted system performance, leading in some cases to apparent data loss at the node level and, more consistently, to a critical increase in latency.

The root cause was **that** the snapshot creation often required more time than the configured election timeout (see Eq. (1)).

$$\exists n \in N : n.electionTimeout > n_{leader}.snapshotTime \quad (1)$$

This was primarily due to the high snapshot threshold, which was set to 60.000 log entries before triggering the compaction process. As a consequence, nodes could become unresponsive for a duration long enough to trigger unnecessary leader election, thereby destabilizing the cluster.

To address this issue, the locking mechanism was removed from the snapshotting phase. Snapshot creation was instead delegated to a dedicated component, allowing it to proceed without holding the main synchronization lock. This change significantly reduced contention, improved system responsiveness, and eliminated the observed instability during high-load scenarios.

Raft RPC Idempotency

A critical challenge in distributed systems based on Raft is ensuring idempotency and providing exactly-once semantics, particularly in the presence of client retries caused by network failures or timeouts. In this implementation, this issue is addressed through an explicit client session tracking mechanism.

Each **LogEntry** is extended to include, in addition to the command, a **clientID** and a strictly increasing **sequenceNum**. These identifiers uniquely represent each client request and allow the system to reliably detect duplicates.

At **run time**, each node maintains an in-memory data structure, referred to as **clientSession**, implemented using a **ConcurrentHashMap**. This structure maps each **clientID** to the highest sequence number that has been successfully processed for that specific client.

When the leader receives a new request through the **propose()** method, it performs a preliminary check against the **clientSession** map. If the incoming **sequenceNum** is less than or equal to the value currently stored for that client, the request is classified as a duplicate, typically generated by network retries. In such cases, the system safely ignores the request while still returning a successful response, thereby avoiding the insertion of redundant entries into the replicated log.

Once a command reaches quorum and is committed, it is applied to the state machine via the **applyLog()** method. At this stage, the corresponding **sequenceNum** in the **clientSession** map is immediately updated, ensuring that any subsequent duplicate requests can be correctly identified and filtered.

To preserve linearizability and idempotency even after log compaction or node restarts, the entire **clientSession** map is serialized as part of the **Snapshot**, together with the current application state. This guarantees that duplicate detection remains consistent across recovery and prevents the re-execution of previously committed operations.

Term Explosion and PreVote Logic

A subtle but critical issue in Raft-based systems is the phenomenon commonly referred to as *term explosion*. This condition arises when nodes repeatedly increment their term without making actual progress in leader election, typically due to network partitions or prolonged communication failures.

In such scenarios, an isolated follower continuously times out, transitions to the candidate state, and increments its term while attempting to start new elections. Since it cannot reach a quorum, each election

inevitably fails, leading to a new timeout and another term increment. Over time, this results in the node accumulating a significantly higher term compared to the rest of the cluster, despite having made no meaningful progress.

While this behavior does not violate Raft’s safety guarantees, it introduces a serious liveness concern. When the partition is resolved, the rejoining node propagates its inflated term to the rest of the cluster. According to Raft’s protocol rules, any node observing a higher term must step down to follower, even if a stable leader is currently active. This can trigger unnecessary leader demotions and cluster-wide elections, temporarily degrading system availability.

To address this issue, an additional validation phase, known as the Pre-Vote mechanism, can be introduced to prevent nodes from incrementing their term unless there is a realistic chance of winning an election.

Election Phase Split-Vote

Another well-known issue in Raft-based systems is the occurrence of *split votes* during leader election. This situation arises when multiple followers transition to the candidate state simultaneously and each one receives a subset of votes, preventing any candidate from reaching the required majority.

Split votes are more likely to occur in clusters where election timeouts overlap or are insufficiently randomized. In such cases, several nodes may start an election at nearly the same time, leading to repeated election failures and temporary unavailability until a leader is eventually elected.

In this implementation, the impact of split votes was observed to be marginal due to the relatively small cluster size (up to 15 nodes) and the use of randomized election timeouts. With a limited number of nodes, the probability of persistent vote fragmentation remains low, and the system typically converges to a new leader within a few election cycles.

When a split vote does occur, the Raft protocol naturally resolves the situation through successive election rounds. Each failed election triggers a new timeout, and due to the randomized delays, it becomes increasingly likely that one node will start the next election earlier than the others, thereby collecting a majority of votes. Overall, while split votes do not compromise safety, they can temporarily impact system liveness. Proper tuning of timing parameters and election strategies is therefore essential to ensure stable leader election behavior under varying network conditions.

Cluster Dimensions

An important design consideration in Raft-based systems is the choice of the cluster size, particularly the preference for an odd number of nodes. While this may appear as a minor detail, it has direct implications on both fault tolerance and the likelihood of split votes.

Raft requires a strict majority (quorum) of nodes to elect a leader and to commit log entries. In a cluster of size N , this quorum is defined as $\lfloor \frac{N}{2} \rfloor + 1$. As a consequence, adding an extra node to move from an odd-sized cluster to an even-sized one does not increase the system’s fault tolerance. For example, both a 3-node and a 4-node cluster can tolerate only a single node failure, as they both require 2 and 3 nodes respectively to form a majority. Therefore, the additional node in the even-sized configuration does not provide any tangible benefit in terms of resilience.

Moreover, even-sized clusters are more prone to split vote scenarios during leader election. In the event of simultaneous candidacy, votes can be evenly divided between competing nodes, preventing any candidate from reaching a majority. While Raft’s randomized election timeouts eventually resolve such situations, the probability of repeated election failures is inherently higher compared to odd-sized clusters, where a strict majority is more easily achieved.

For these reasons, it is generally recommended to deploy Raft clusters with an odd number of nodes. This choice maximizes fault tolerance efficiency while reducing the likelihood of election instability, leading to

more predictable and robust system behavior.

Write-Ahead Log

A fundamental component of the system is the Write-Ahead Log (WAL), introduced to guarantee durability and crash recovery in the presence of node failures. In a distributed consensus algorithm such as Raft, it is essential that log entries are not only replicated across nodes, but also safely persisted to stable storage before being considered committed.

The WAL ensures that every `LogEntry` is written to persistent storage before being applied to the state machine. This design prevents data loss in the event of crashes, as nodes can reconstruct their state by replaying the log upon restart. In this implementation, the WAL acts as the authoritative source of truth for the replicated state, allowing the system to recover consistently even after abrupt interruptions.

One of the main advantages of this approach is the strong durability guarantee it provides. By persisting entries before acknowledgment, the system ensures that once a command is committed, it cannot be lost, even under severe failure scenarios. Additionally, the WAL integrates naturally with Raft's log replication model, simplifying recovery logic and maintaining consistency across nodes.

However, the introduction of the WAL also exposes several performance and architectural trade-offs. The most notable drawback is the latency introduced by synchronous disk I/O operations. Writing each log entry to stable storage can significantly slow down the system, particularly under high request throughput or when running on non-optimized storage devices. This overhead becomes even more evident when combined with other blocking operations, potentially leading to contention issues.

Furthermore, the interaction between the WAL and the concurrency model must be carefully managed. As observed during testing, naive synchronization strategies around disk writes can introduce bottlenecks, especially when combined with other critical sections such as snapshotting. This can negatively impact responsiveness and, in extreme cases, interfere with timing-sensitive mechanisms like leader election.

Overall, while the Write-Ahead Log is essential to ensure correctness and durability, its integration requires careful tuning and architectural considerations. Balancing persistence guarantees with performance remains a key challenge, particularly in high-concurrency environments.

Was Java 21 a Good Solution?

From an academic and experimental perspective, the choice of Java 21 proved to be both appropriate and insightful, particularly due to the introduction of Virtual Threads through Project Loom. This feature enabled a straightforward and expressive implementation of concurrency, allowing the system to adopt a thread-per-request model without incurring the traditional overhead associated with OS-level threads. As a result, the implementation remained relatively simple, readable, and easier to reason about compared to more complex asynchronous or reactive paradigms.

However, despite these advantages, it is important to acknowledge the inherent limitations of the JVM in the context of distributed consensus systems. Java's garbage collection, while highly optimized, introduces non-deterministic pauses that can interfere with timing-sensitive mechanisms such as leader election. Additionally, issues such as thread pinning during blocking I/O operations can reduce the effectiveness of Virtual Threads, leading to potential performance bottlenecks under high load. These characteristics make Java less predictable in environments where strict timing guarantees are desirable.

In contrast, Go offers a concurrency model based on goroutines that is conceptually similar to Virtual Threads but more mature and deeply integrated into the runtime. Its lightweight scheduler and low-latency garbage collector provide a good balance between performance and developer productivity. This makes Go a popular choice for production-grade distributed systems, as evidenced by projects such as etcd and Consul. However, Go trades off fine-grained control over memory management and may still exhibit latency spikes

due to garbage collection, albeit typically less pronounced than in JVM-based systems.

Rust, on the other hand, represents a fundamentally different approach by eliminating garbage collection entirely and enforcing memory safety at compile time. This results in highly predictable performance and the absence of runtime pauses, making it particularly well-suited for systems requiring strict control over latency and resource usage. Furthermore, Rust allows for low-level optimizations that can significantly improve throughput and efficiency. The downside, however, lies in its increased complexity and steeper learning curve, which can slow down development and make the implementation of complex distributed protocols more challenging.

In conclusion, while Java 21 with Virtual Threads provides an excellent platform for prototyping and educational purposes, offering clarity and ease of development, languages such as Go and Rust are often better suited for production environments where performance predictability, low latency, and fine-grained control over system resources are critical requirements.